

Lista de Exercícios – Agentes, ambientes e NumPy

Parte 1 – PEAS e Propriedades do Ambiente

- Q1.** Para cada um dos agentes abaixo, preencha a estrutura PEAS (Performance, Environment, Actuators, Sensors):
- (a) Um robô aspirador de pó (2 cômodos)
 - (b) Um programa que joga xadrez
 - (c) Um carro autônomo
 - (d) Um sistema de recomendação de filmes (Netflix)
- Q2.** Classifique cada um dos ambientes acima segundo as 6 propriedades (observável/parcialmente observável, determinístico/estocástico, episódico/sequencial, estático/dinâmico, discreto/contínuo, agente único/multiagente). Justifique cada classificação em uma frase.
- Q3.** O GridWorld 4×4 implementado em aula é um ambiente com as seguintes propriedades: totalmente observável, determinístico, sequencial, estático, discreto, agente único.
- (a) O que mudaria na implementação de `step()` se o ambiente fosse **estocástico** (ex.: 20% de chance de o movimento falhar e o agente não sair do lugar)?
 - (b) O que mudaria se fosse **parcialmente observável** (ex.: o agente só enxerga as células adjacentes)?
 - (c) O que mudaria se fosse **multiagente** (ex.: dois agentes competindo para alcançar o objetivo primeiro)?

Parte 2 – Tipos de Agente

- Q4.** Descreva a diferença entre um agente **reativo simples** e um agente **com estado interno**. Dê um exemplo de uma situação em que o agente reativo falha e o agente com estado interno resolve.
- Q5.** Explique por que um agente **baseado em objetivo** precisa de **busca** para decidir. Qual é a relação entre o espaço de estados e a função `decidir()`?
- Q6.** O diagrama do **agente que aprende** (agente + crítico + elemento de aprendizado) é apresentado em aula como “o diagrama do reinforcement learning”. Explique o papel de cada componente e como eles interagem.

Parte 3 – NumPy e GridWorld

- Q7.** Considere o seguinte código:

```
import numpy as np

a = np.array([10, 20, 30, 40])
b = np.array([1, 2, 3, 4])
```

Qual é o resultado de cada operação abaixo? (Responda sem executar – depois confira no Python.)

- (a) `a + b`
- (b) `a * b`
- (c) `a > 25`
- (d) `a[a > 25]`
- (e) `np.argmax(a)`
- (f) `np.sum(a * b)`

Q8. O GridWorld usa `np.array` para representar a posição do agente. Por que usar um array NumPy em vez de uma tupla ou lista Python? Dê duas razões.

Q9. Desafio de vetorização: O código abaixo calcula a distância euclidiana entre um ponto e todos os pontos de uma lista usando laço `for`:

```
import math

def distancias_loop(p, pontos):
    return [math.sqrt((p[0]-q[0])**2 + (p[1]-q[1])**2)
            for q in pontos]
```

Reescreva esta função usando NumPy vetorizado. Dica: use `np.linalg.norm` com o parâmetro `axis`.

Q10. Implementação: Modifique o GridWorld da aula para incluir:

- (a) Um parâmetro `prob_falha` no `__init__` que, se > 0 , faz com que a ação escolhida tenha probabilidade `prob_falha` de resultar em um movimento aleatório (ambiente estocástico).
- (b) Um método `render()` que imprime a grade no terminal com caracteres: `.` para célula vazia, `#` para obstáculo, `G` para objetivo, `A` para agente.
- (c) Teste seu GridWorld estocástico com `prob_falha=0.3` e execute o agente reativo 10 vezes. A taxa de sucesso (chegar ao objetivo) muda?

Parte 4 – Conexões

Q11. O GridWorld que você implementou esta semana será reutilizado nas Semanas 10–14 para reinforcement learning. Explique: o que precisará mudar no código do GridWorld para que ele suporte Q-learning? O que **não** precisará mudar?

Q12. Ponte para o Deep Learning: O artigo “Human-level control through deep reinforcement learning” (Mnih et al., 2015, Nature) – que introduziu o DQN – usa a interface `reset()/step()` da biblioteca Arcade Learning Environment (ALE). Compare esta interface com a do GridWorld. O que há de fundamentalmente igual? O que há de diferente na escala?